

Assignment 6 Hanliang Xu

1. erase rule:

$$\langle e, s \rangle \Rightarrow \langle 0, \emptyset \rangle$$

2. Use CP for checkpoint and SB for snapback

function application:

$$\langle e_1, S_1, C_1 \rangle \Rightarrow \langle \text{fun } x \rightarrow e, S_2, C_2 \rangle, \langle e_2, S_2, C_2 \rangle \Rightarrow \langle v_2, S_3, C_3 \rangle, \langle e[v_2/x], S_3, C_3 \rangle \Rightarrow \langle v, S_4, C_4 \rangle$$

$$\langle e_1 e_2, S_1, C_1 \rangle \Rightarrow \langle v, S_4, C_4 \rangle$$

assignment := :

$$\langle e_1, S_1, C_1 \rangle \Rightarrow \langle c, S_2, C_2 \rangle, \langle e_2, S_2, C_2 \rangle \Rightarrow \langle v, S_3, C_3 \rangle$$

$$\langle e_1 := e_2, S_1, C_1 \rangle \Rightarrow \langle v, S_3 \{ C \mapsto v \}, C_3 \rangle$$

Checkpoint :

$$\langle \text{Checkpoint}, S, S' \rangle \Rightarrow \langle 0, S, S \rangle$$

Snapback :

$$S'' = S' \cup \{ (l \mapsto S(l)) \mid l \notin \text{Dom}(S') \}$$

$$\langle \text{Snapback}, S_1, S' \rangle \Rightarrow \langle 0, S'', S' \rangle$$

3. a. operational equivalence for FbSRX:

$e \cong e'$ if and only if for all contexts C such that $C[e]$ and $C[e']$ are both closed expressions, it holds that starting from a fixed initial state S_0 , $\langle C[e], S_0 \rangle \Rightarrow \langle v, S_1 \rangle$ for some v if and only if $\langle C[e'], S_0 \rangle \Rightarrow \langle v', S_1' \rangle$ for some v' , where v and v' can either be a normal value or an exception.

b. Let $C \stackrel{\text{def}}{=} \text{Let } x = 0 \text{ In } (\text{If } \bullet \text{ Then } 0 \text{ Else } (0 \ 0)).$

$$\langle C[\text{Try True with } \# \text{wipeout}(x) \rightarrow \text{False}], \emptyset \rangle \Rightarrow \langle 0, \emptyset \rangle,$$

while $\langle C[\text{False}], \emptyset \rangle \not\Rightarrow \langle v, \emptyset \rangle$ for any v .

Thus, by definition, $\text{Try True with } \# \text{wipeout}(x) \rightarrow \text{False} \not\equiv \text{False}$

c. Let $C \stackrel{\text{def}}{=} \text{If } (\text{Try } \bullet \text{ With } \# \text{wipeout}(x) \rightarrow x) = 4 \text{ Then } 0 \text{ Else } (0 \ 0).$

$$\langle C[(\text{Raise } \# \text{wipeout}(4)) + 4], \emptyset \rangle \Rightarrow \langle 0, \emptyset \rangle,$$

while $\langle C[7 - (\text{Raise } \# \text{wipeout}(5))], \emptyset \rangle \not\Rightarrow \langle v, \emptyset \rangle$ for any v .

Thus, by definition, $(\text{Raise } \# \text{wipeout}(4)) + 4 \not\equiv 7 - (\text{Raise } \# \text{wipeout}(5))$

d. are operational equivalent

e. are operational equivalent

4. a. + operation run-time type mismatch exception

$e_1 \Rightarrow v_1, v_1 \notin \text{numbers}, (\text{don't run } e_2)$

$e_1 + e_2 \Rightarrow \text{Raise } \#TypeErr(0)$

$e_1 \Rightarrow v_1, v_1 \in \text{numbers}, e_2 \Rightarrow v_2, v_2 \notin \text{numbers}$

$e_1 + e_2 \Rightarrow \text{Raise } \#TypeErr(0)$

b. Let safeadd = Function $x \rightarrow$ Function $y \rightarrow$
Try $(x + y)$ With $\#TypeErr(0) \rightarrow 0$

5. a. Let pointImpl = {

x = Ref 4;

y = Ref 3;

magnitude = Fun this $\rightarrow$ Fun dummy $\rightarrow !(\text{this.x}) + !(\text{this.y})$;

iszero = Fun this $\rightarrow$ Fun dummy $\rightarrow (\text{this.magnitude this } \{\}) = 0$;

setx = Fun this $\rightarrow$ Fun newX $\rightarrow \text{point.x} = \text{newX}$;

sety = Fun this $\rightarrow$ Fun newY $\rightarrow \text{point.y} = \text{newY}$

} In point = {

magnitude = pointImpl.magnitude pointImpl;

iszero = pointImpl.iszero pointImpl;

setx = pointImpl.setx pointImpl;

sety = pointImpl.sety pointImpl

} In point.magnitude {}

b. Let prePoint =

Let privateThis = {

x = Ref 4;

y = Ref 3

} In Fun this $\rightarrow$ {

magnitude = Fun dummy $\rightarrow !(\text{privateThis.x}) + !(\text{privateThis.y})$;

iszero = Fun dummy $\rightarrow (\text{this.magnitude } \{\}) = 0$;

setx = Fun newX $\rightarrow \text{privateThis.x} := \text{newX}$;

sety = Fun newY $\rightarrow \text{privateThis.y} := \text{newY}$

} In Let point = prePoint prePoint In point.magnitude {}